

York University
Lassonde School of Engineering

EECS3311 – Software Design

Market Intelligence Agent

Stage 1 – Software and Agent Design

Student: [Erdem Calikoglu]
Student Number: [218188771]
Instructor: [Song Wang]
Date: [September 25, 2026]

Contents

1	Project Description and Scope	3
1.1	Project Title	3
1.2	Project Description	3
2	Problem	3
2.1	Intended Users	3
2.2	Current Problem	4
2.3	Why an Agent Is Appropriate	4
3	Major Features	5
3.1	User Authentication	5
3.2	Persistent User Profiles	5
3.3	Natural-Language Interaction	5
3.4	Terminal Command Interaction	5
3.5	Agent Planning and Tool Selection	6
3.6	Market-Data Retrieval	6
3.7	ETF Exposure Analysis	6
3.8	Security Comparison	6
3.9	ETF Holding Overlap	7
3.10	Financial-News Retrieval	7
3.11	Market-Movement Explanation	7
3.12	Portfolio Management	7
3.13	Watchlist Management	8
3.14	Portfolio Exposure Analysis	8
3.15	Conversation Context	8
3.16	Persistent Conversation Memory	8
3.17	Validation and Failure Handling	8
4	Major User Interactions	9
5	Use Cases	9
5.1	UC-01 – Register Account	9
5.2	UC-02 – Log In	10
5.3	UC-03 – Log Out	10
5.4	UC-04 – Get Market Quote	10
5.5	UC-05 – Get Security Overview	11
5.6	UC-06 – Analyze ETF Exposure	11
5.7	UC-07 – Compare Securities	12
5.8	UC-08 – Analyze ETF Holding Overlap	12
5.9	UC-09 – Retrieve Financial News	13
5.10	UC-10 – Explain Recent Market Movement	13
5.11	UC-11 – Add Portfolio Position	14
5.12	UC-12 – Remove Portfolio Position	14
5.13	UC-13 – View Portfolio	14
5.14	UC-14 – Manage Watchlist	15
5.15	UC-15 – Analyze Portfolio Exposure	15
5.16	UC-16 – Ask a Contextual Follow-Up Question	16

6	UML Use-Case Diagram	16
7	Sequence Diagrams	18
7.1	UC-04 – Get Market Quote	18
7.2	UC-06 – Analyze ETF Exposure	18
7.3	UC-10 – Explain Recent Market Movement	19
7.4	UC-15 – Analyze Portfolio Exposure	20
7.5	UC-16 – Contextual Follow-Up	21
8	Class Design	22
8.1	User Interface Layer	22
8.2	Authentication and Session Layer	23
8.3	Agent Orchestration Layer	23
8.4	Agent Tool Layer	23
8.5	Domain Service Layer	24
8.6	External Provider Layer	24
8.7	Persistence Layer	24
9	UML Class Diagrams	25
9.1	Core Agent Architecture	25
9.2	Authentication and Persistence	27
9.3	Domain Model	29
10	Feature-to-Design Mapping	31
11	Important Design Decisions	32
11.1	Terminal-Based User Interface	32
11.2	Separation of Deterministic and LLM Behavior	32
11.3	Controlled Agent Execution	33
11.4	Agent Tool Abstraction	33
11.5	Provider Abstraction and Dependency Inversion	33
11.6	Repository Abstraction	34
11.7	Multi-User Data Isolation	34
11.8	SQLite Persistence	34
11.9	Password Hashing Abstraction	35
11.10	Financial Values and BigDecimal	35
11.11	Constructor-Based Dependency Injection	35
11.12	Structured Failure Results	35
12	Design Traceability	36
13	Stage 1 Summary	36

1 Project Description and Scope

1.1 Project Title

Market Intelligence Agent

1.2 Project Description

The Market Intelligence Agent is a Java-based, multi-user AI agent designed to help users retrieve, analyze, and understand financial-market information through natural-language interaction and a professional terminal-style interface.

Financial information is commonly distributed across several independent sources. A user interested in a stock or exchange-traded fund (ETF) may need to separately retrieve current market prices, inspect ETF holdings, calculate sector or geographic exposure, search for relevant financial news, and determine how the security contributes to an existing portfolio.

The proposed system combines these activities within a single agent-based application.

The system will combine traditional deterministic Java components with a Large Language Model (LLM), external financial-data services, persistent storage, user authentication, and agent capabilities such as planning, tool selection, retrieval, contextual memory, and multi-step task execution.

The application will support multiple users. Each registered user will have an independent account containing a portfolio, watchlist, preferences, and conversation history. User-specific information will be isolated from other users and persisted between application sessions.

The primary user interface will be a terminal-style interface inspired by professional financial terminals. Users will be able to interact with the system using both natural-language requests and explicit terminal commands.

The project is not intended to function as a brokerage service or autonomous trading platform. Its scope is limited to financial-information retrieval, ETF and portfolio analysis, market-news retrieval, watchlist and portfolio management, and AI-assisted explanation of retrieved information.

2 Problem

2.1 Intended Users

The intended users are retail investors, students, and financially interested users who want to retrieve and understand market information without manually navigating several independent financial-information services.

Typical users may wish to:

- monitor stocks and ETFs;
- maintain a personal watchlist;
- maintain a simple investment portfolio;
- examine ETF sector or geographic exposure;
- compare securities or ETFs;
- identify ETF holding overlap;
- retrieve recent financial news;
- understand recent market movements;

- analyze portfolio exposure; and
- ask contextual follow-up questions using natural language.

2.2 Current Problem

Even relatively simple investment questions may require the user to combine information from several sources.

For example, consider the request:

“What sectors am I most exposed to, and has there been any important recent news affecting my largest positions?”

Answering this manually may require the user to:

1. retrieve the current portfolio;
2. identify direct stock and ETF positions;
3. retrieve the underlying holdings of ETF positions;
4. calculate aggregate sector exposure;
5. identify the largest contributing holdings;
6. retrieve recent news relating to those holdings; and
7. combine the information into an understandable summary.

Traditional financial applications can provide much of the raw information, but the user is still responsible for selecting the appropriate tools, executing the required operations, and combining the results.

2.3 Why an Agent Is Appropriate

An agent-based architecture is appropriate because different user requests require different combinations of software components and external tools.

For example:

“What is the latest price of NVDA?”

requires market-data retrieval.

However:

“Why has NVDA been moving recently?”

may require both market information and recent financial news.

A portfolio-level question may require additional operations, including portfolio retrieval, ETF holdings retrieval, and deterministic exposure calculations.

The agent will therefore:

1. interpret the user’s request;
2. identify the required task;
3. create an execution plan;
4. select appropriate tools;
5. execute those tools;
6. validate the returned information;

7. perform deterministic calculations where required; and
8. construct a user-facing response.

The LLM will not be responsible for deterministic business logic such as authentication, portfolio modification, financial calculations, persistence, or input validation. These operations will be implemented directly in Java.

3 Major Features

3.1 User Authentication

The system will support account registration, login, and logout.

Passwords will not be stored directly. A secure password hash will be stored in the application's persistent database.

Authentication allows portfolio information, watchlists, preferences, and conversation history to remain isolated between different users.

3.2 Persistent User Profiles

Each registered user will have independent persistent information including:

- portfolio positions;
- watchlist securities;
- user preferences;
- conversation sessions; and
- conversation messages.

This information will remain available after the application is restarted.

3.3 Natural-Language Interaction

Users will be able to communicate with the system using natural-language requests.

Example requests include:

“What is Microsoft trading at?”

“Analyze XEQT’s sector exposure.”

“How much overlap is there between XEQT and VFV?”

“Why has NVIDIA been moving recently?”

The system will interpret the request and determine the appropriate operations.

3.4 Terminal Command Interaction

In addition to natural language, deterministic commands will provide direct access to frequently used functionality.

Possible commands include:

`/portfolio`

```
/watchlist  
/quote NVDA  
/news MSFT  
/compare XEQT VFV  
/history  
/logout
```

This allows predictable command-based interaction alongside flexible natural-language interaction.

3.5 Agent Planning and Tool Selection

The agent will analyze each natural-language request and determine which tools are required.

A simple request may require only one tool, while a complex request may require several tools and multiple execution steps.

The planning component will produce a structured execution plan. The Java application will validate and execute this plan rather than allowing the LLM to directly perform arbitrary application operations.

3.6 Market-Data Retrieval

The system will retrieve financial-market information through an external market-data provider.

Retrieved information may include:

- security symbol;
- security name;
- latest available price;
- previous closing price;
- percentage change;
- currency;
- timestamp; and
- selected security metadata.

3.7 ETF Exposure Analysis

The system will retrieve ETF holdings and perform deterministic exposure calculations.

Supported analysis may include:

- underlying holdings;
- holding weights;
- sector exposure;
- geographic exposure;
- overlapping holdings; and
- portfolio-level indirect ETF exposure.

3.8 Security Comparison

Users will be able to compare two supported securities.

Depending on the security type and available information, the comparison may include:

- market information;
- security metadata;
- ETF holdings;
- sector exposure;
- geographic exposure; and
- holding overlap.

3.9 ETF Holding Overlap

The system will retrieve holdings for two ETFs, identify common holdings, and calculate a deterministic overlap result.

The LLM may explain the result but will not perform the overlap calculation.

3.10 Financial-News Retrieval

The system will retrieve recent financial news relating to a requested security.

Article information may include:

- headline;
- source;
- publication time;
- related symbols; and
- article URL.

The LLM may summarize retrieved news but will not generate articles that were not returned by the configured news provider.

3.11 Market-Movement Explanation

The system will combine market information and recent retrieved news when a user asks about recent price movement.

The final response will distinguish retrieved information from generated explanation.

If supporting information cannot be retrieved, the system will report the limitation rather than silently fabricating a cause.

3.12 Portfolio Management

Users will be able to:

- add portfolio positions;
- remove portfolio positions;
- view the current portfolio; and
- use portfolio data in agent analysis.

Each portfolio belongs to exactly one user account.

3.13 Watchlist Management

Each user will maintain an independent watchlist.

Users will be able to add securities, remove securities, and view the watchlist.

The watchlist may also be used as context for market-data or news requests.

3.14 Portfolio Exposure Analysis

The system will calculate portfolio-level sector or geographic exposure.

For ETF positions, the application may retrieve the ETF’s underlying holdings before performing the calculation.

Financial calculations will be performed using deterministic Java components rather than the LLM.

3.15 Conversation Context

The application will maintain short-term context during a conversation.

For example:

User: “Analyze XEQT.”

User: “What about its geographic exposure?”

The application should use the recent conversation to determine that “its” refers to XEQT.

3.16 Persistent Conversation Memory

Conversation history will be associated with the authenticated user and stored in persistent storage.

This allows previous messages and sessions to remain accessible after the application is restarted.

Persistent memory will be managed by deterministic Java components rather than allowing the LLM to independently determine what should be permanently stored.

3.17 Validation and Failure Handling

The system will explicitly handle failure situations including:

- malformed ticker symbols;
- invalid portfolio quantities;
- incorrect authentication credentials;
- duplicate usernames;
- unavailable APIs;
- incomplete external responses;
- missing ETF holdings;
- invalid LLM-generated plans;
- unavailable tools; and
- ambiguous contextual requests.

Failures will be represented using structured results wherever practical.

4 Major User Interactions

The system is designed to support sixteen major user-facing interactions.

1. UC-01 – Register Account
2. UC-02 – Log In
3. UC-03 – Log Out
4. UC-04 – Get Market Quote
5. UC-05 – Get Security Overview
6. UC-06 – Analyze ETF Exposure
7. UC-07 – Compare Securities
8. UC-08 – Analyze ETF Holding Overlap
9. UC-09 – Retrieve Financial News
10. UC-10 – Explain Recent Market Movement
11. UC-11 – Add Portfolio Position
12. UC-12 – Remove Portfolio Position
13. UC-13 – View Portfolio
14. UC-14 – Manage Watchlist
15. UC-15 – Analyze Portfolio Exposure
16. UC-16 – Ask a Contextual Follow-Up Question

5 Use Cases

5.1 UC-01 – Register Account

Primary Actor: User

Goal: Create a new user account.

Preconditions: The application is running and persistent storage is available.

Trigger: The user selects account registration.

Main Success Scenario:

1. The user selects the registration option.
2. The system requests a username and password.
3. The user provides the requested credentials.
4. The system validates the username and password.
5. The system verifies that the username is not already registered.
6. The password is passed to the configured password-hashing component.
7. The resulting password hash is stored with the new user.
8. The system confirms successful registration.

Alternative/Exception Flows:

- If the username already exists, registration is rejected.
- If the credentials fail validation, the user must provide valid input.
- If persistence fails, the account is not created and the error is reported.

Postconditions: A new user account exists in persistent storage.

5.2 UC-02 – Log In

Primary Actor: User

Goal: Authenticate and access user-specific information.

Preconditions: The user has previously registered.

Trigger: The user submits a username and password.

Main Success Scenario:

1. The user enters a username and password.
2. **AuthService** retrieves the matching user.
3. The submitted password is checked against the stored password hash.
4. Authentication succeeds.
5. **SessionManager** creates a new user session.
6. User-specific portfolio, watchlist, preferences, and memory become available.
7. The terminal displays the authenticated home screen.

Alternative/Exception Flows:

If authentication fails, the system rejects the request without indicating whether the username or password was incorrect.

Postconditions: An authenticated user session exists.

5.3 UC-03 – Log Out

Primary Actor: Authenticated User

Goal: End the active authenticated session.

Preconditions: A user is authenticated.

Trigger: The user enters the logout command.

Main Success Scenario:

1. The user requests logout.
2. Required user state is persisted.
3. The current session is invalidated.
4. User-specific in-memory session state is cleared.
5. The application returns to the login screen.

Postconditions: No authenticated user session remains active.

5.4 UC-04 – Get Market Quote

Primary Actor: Authenticated User

Goal: Retrieve market information for a security.

Preconditions: The market-data provider is configured.

Trigger: The user requests a quote using either natural language or a terminal command.

Main Success Scenario:

1. The user submits a quote request.
2. The request is interpreted.
3. The requested symbol is validated.
4. The market-data tool is selected.
5. The external market-data provider is queried.
6. The returned information is validated.
7. A structured **MarketQuote** is created.
8. The result is formatted and displayed.

Alternative/Exception Flows:

- An invalid symbol is rejected before API execution.
- If the provider is unavailable, the system reports that current information could not be retrieved.

Postconditions: The user receives retrieved market information or an explicit failure result.

5.5 UC-05 – Get Security Overview

Primary Actor: Authenticated User

Goal: Retrieve general information about a security.

Preconditions: The market-data provider is configured.

Trigger: The user requests an overview of a security.

Main Success Scenario:

1. The requested symbol is identified.
2. The symbol is validated.
3. **MarketDataTool** requests security metadata.
4. The provider returns the available security information.
5. The information is mapped to a **SecurityOverview**.
6. The response is formatted and displayed.

Postconditions: The user receives an overview based on retrieved information.

5.6 UC-06 – Analyze ETF Exposure

Primary Actor: Authenticated User

Goal: Understand the sector or geographic composition of an ETF.

Trigger: The user requests ETF exposure analysis.

Main Success Scenario:

1. The request is interpreted as an ETF exposure task.

2. The requested ETF symbol is validated.
3. `ETFExposureTool` retrieves ETF holdings.
4. External holdings are converted into `ETFHolding` objects.
5. `ExposureCalculator` calculates the requested exposure.
6. An `ExposureResult` is produced.
7. The deterministic result is passed to the response composer.
8. The LLM generates an explanation based on the supplied result.
9. The analysis is displayed.

Alternative/Exception Flows:

If sufficient holding information is unavailable, the system reports that the requested exposure cannot be calculated from the available data.

Postconditions: A deterministic exposure result is produced when sufficient information is available.

5.7 UC-07 – Compare Securities

Primary Actor: Authenticated User

Goal: Compare two supported securities.

Trigger: The user requests a comparison between two symbols.

Main Success Scenario:

1. Both symbols are extracted and validated.
2. The planner determines which information is required.
3. The appropriate market-data and/or ETF tools are executed.
4. Retrieved results are normalized into comparable structures.
5. `ComparisonTool` constructs the comparison.
6. The response composer presents the results to the user.

Alternative/Exception Flows:

If one symbol cannot be retrieved, the comparison does not silently substitute unsupported data.

Postconditions: A structured comparison is displayed.

5.8 UC-08 – Analyze ETF Holding Overlap

Primary Actor: Authenticated User

Goal: Identify overlapping holdings between two ETFs.

Trigger: The user requests ETF overlap analysis.

Main Success Scenario:

1. Both ETF symbols are identified and validated.
2. ETF holdings are retrieved for the first ETF.
3. ETF holdings are retrieved for the second ETF.
4. `ExposureCalculator` identifies common holdings.

5. The overlap percentage is calculated.
6. An `OverlapResult` is produced.
7. The result is explained and displayed.

Alternative/Exception Flows:

If holding information for either ETF is unavailable, the overlap calculation is not performed and the missing information is reported.

Postconditions: The user receives deterministic overlap information.

5.9 UC-09 – Retrieve Financial News

Primary Actor: Authenticated User

Goal: Retrieve recent financial news relating to a security.

Trigger: The user requests recent news.

Main Success Scenario:

1. The requested security is identified.
2. The symbol is validated.
3. `NewsTool` creates a retrieval request.
4. `NewsProvider` retrieves recent articles.
5. Returned articles are converted into `NewsArticle` objects.
6. Retrieved information is passed to the response composer.
7. The LLM summarizes the retrieved articles.
8. Article metadata is included in the result.

Alternative/Exception Flows:

If no suitable articles are retrieved, the system reports that no relevant recent news was available.

Postconditions: The final response is based on successfully retrieved news information.

5.10 UC-10 – Explain Recent Market Movement

Primary Actor: Authenticated User

Goal: Understand information potentially associated with recent security price movement.

Trigger: The user requests an explanation for recent movement.

Main Success Scenario:

1. The planner determines that market data and news are required.
2. `MarketDataTool` retrieves current or recent market information.
3. `NewsTool` retrieves recent related news.
4. Results from both tools are validated.
5. The structured information is passed to `ResponseComposer`.
6. The LLM produces an explanation constrained to the supplied data.
7. The result is displayed.

Alternative/Exception Flows:

If one source fails, the response identifies the unavailable information.

The system must not invent a causal explanation when supporting information has not been retrieved.

Postconditions: The user receives a synthesis of retrieved market and news information.

5.11 UC-11 – Add Portfolio Position

Primary Actor: Authenticated User

Goal: Add a position to the user's portfolio.

Trigger: The user requests a portfolio addition.

Main Success Scenario:

1. The system obtains the symbol and quantity.
2. The symbol and quantity are validated.
3. A `PortfolioPosition` is created.
4. `PortfolioService` loads the current user's portfolio.
5. The position is added.
6. `PortfolioRepository` persists the updated portfolio.
7. The system confirms the change.

Postconditions: The new position belongs to the authenticated user's stored portfolio.

5.12 UC-12 – Remove Portfolio Position

Primary Actor: Authenticated User

Goal: Remove a security from the user's portfolio.

Trigger: The user requests position removal.

Main Success Scenario:

1. The requested symbol is identified.
2. The current user's portfolio is loaded.
3. The matching position is located.
4. The position is removed.
5. The updated portfolio is persisted.
6. The system confirms removal.

Alternative/Exception Flows:

If the requested position does not exist, the portfolio remains unchanged and the user is informed.

Postconditions: The requested position no longer exists when removal succeeds.

5.13 UC-13 – View Portfolio

Primary Actor: Authenticated User

Goal: View the current user's portfolio.

Preconditions: The user is authenticated.

Trigger: The user requests the portfolio.

Main Success Scenario:

1. `SessionManager` identifies the authenticated user.
2. `PortfolioService` requests that user's portfolio.
3. `PortfolioRepository` loads the portfolio.
4. Positions are formatted for terminal display.
5. The portfolio is displayed.

Postconditions: The stored portfolio remains unchanged.

5.14 UC-14 – Manage Watchlist

Primary Actor: Authenticated User

Goal: Add, remove, or view watchlist securities.

Trigger: The user performs a watchlist operation.

Main Success Scenario:

1. The requested watchlist action is identified.
2. If necessary, the supplied symbol is validated.
3. `WatchlistService` loads the current user's watchlist.
4. The requested operation is performed.
5. The updated watchlist is persisted when modified.
6. The current watchlist is displayed.

Postconditions: The user's watchlist reflects the successful requested change.

5.15 UC-15 – Analyze Portfolio Exposure

Primary Actor: Authenticated User

Goal: Calculate portfolio-level exposure.

Trigger: The user requests portfolio exposure analysis.

Main Success Scenario:

1. The authenticated user's portfolio is loaded.
2. Direct stock positions are identified.
3. ETF positions are identified.
4. ETF holdings are retrieved where required.
5. Position quantities and ETF weights are combined.
6. `ExposureCalculator` calculates aggregate exposure.
7. An `ExposureResult` is produced.
8. The deterministic result is supplied to the response composer.

9. The LLM explains the calculated result.
10. The analysis is displayed.

Alternative/Exception Flows:

If information for an ETF cannot be retrieved, the system identifies the missing position and does not represent incomplete analysis as complete.

Postconditions: The user receives calculated portfolio exposure information.

5.16 UC-16 – Ask a Contextual Follow-Up Question

Primary Actor: Authenticated User

Goal: Continue an existing conversation without repeating previously specified entities.

Preconditions: Relevant conversation history exists.

Trigger: The user submits a request containing a contextual reference.

Main Success Scenario:

1. The new message is received.
2. `MemoryService` retrieves recent conversation context.
3. The relevant previous entity is identified.
4. The contextual request is resolved.
5. `RequestPlanner` creates an execution plan.
6. Required tools are executed.
7. The response is generated.
8. The user message and agent response are persisted.

Alternative/Exception Flows:

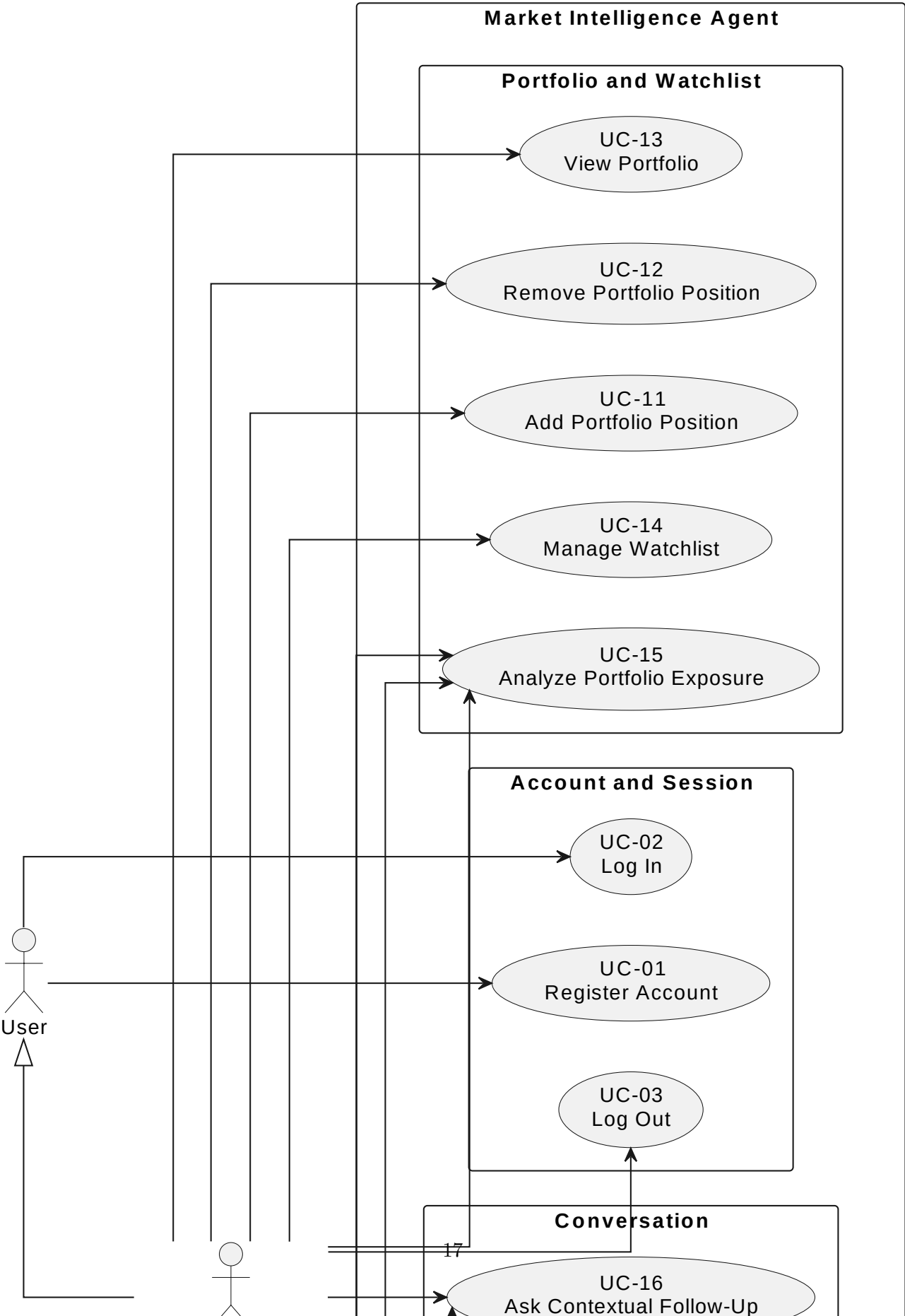
If the contextual reference is ambiguous, the system requests clarification rather than assuming an entity.

Postconditions: Conversation history contains the completed interaction.

6 UML Use-Case Diagram

Figure 1 presents the major user-facing interactions and external systems used by the Market Intelligence Agent.

Market Intelligence Agent - Use-Case Diagram



7 Sequence Diagrams

Representative sequence diagrams are used to show how the major software components collaborate during different types of agent interactions.

7.1 UC-04 – Get Market Quote

Figure 2 demonstrates a relatively simple agent workflow involving request interpretation, validation, tool selection, and external market-data retrieval.

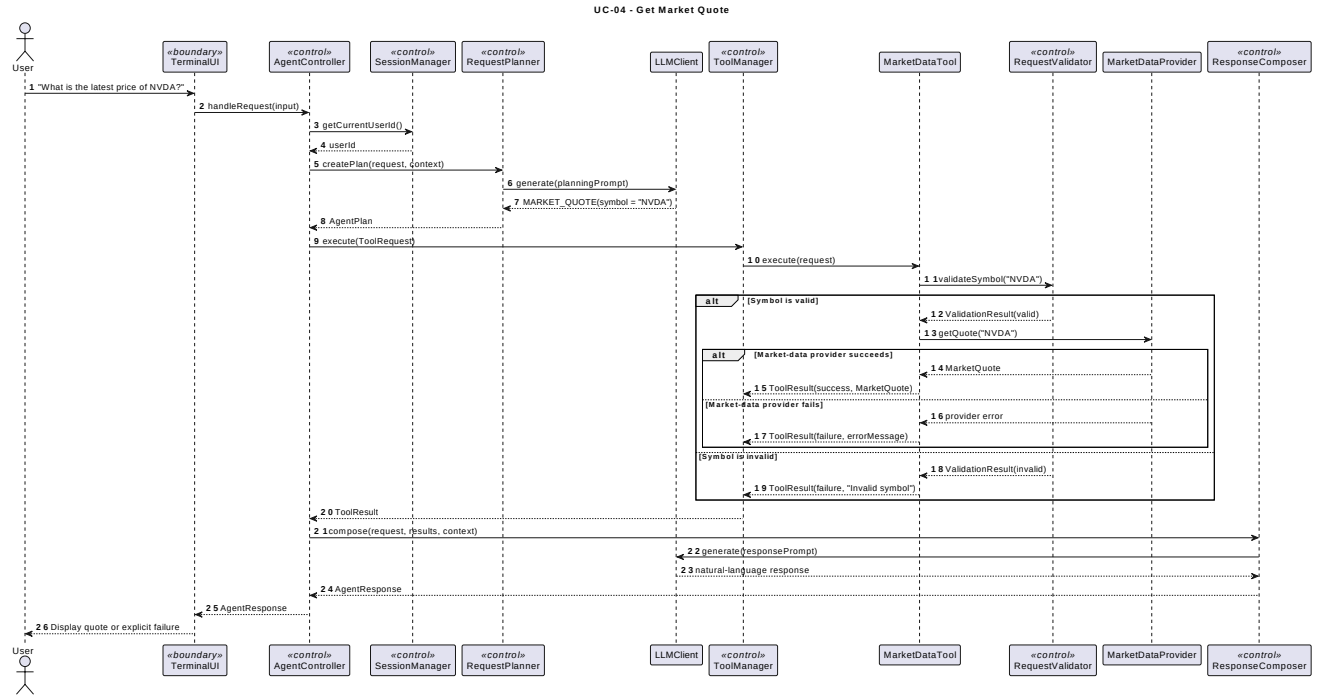


Figure 2: Sequence Diagram for UC-04 – Get Market Quote

7.2 UC-06 – Analyze ETF Exposure

Figure 3 demonstrates the separation between ETF data retrieval, deterministic financial calculation, and LLM-based explanation.

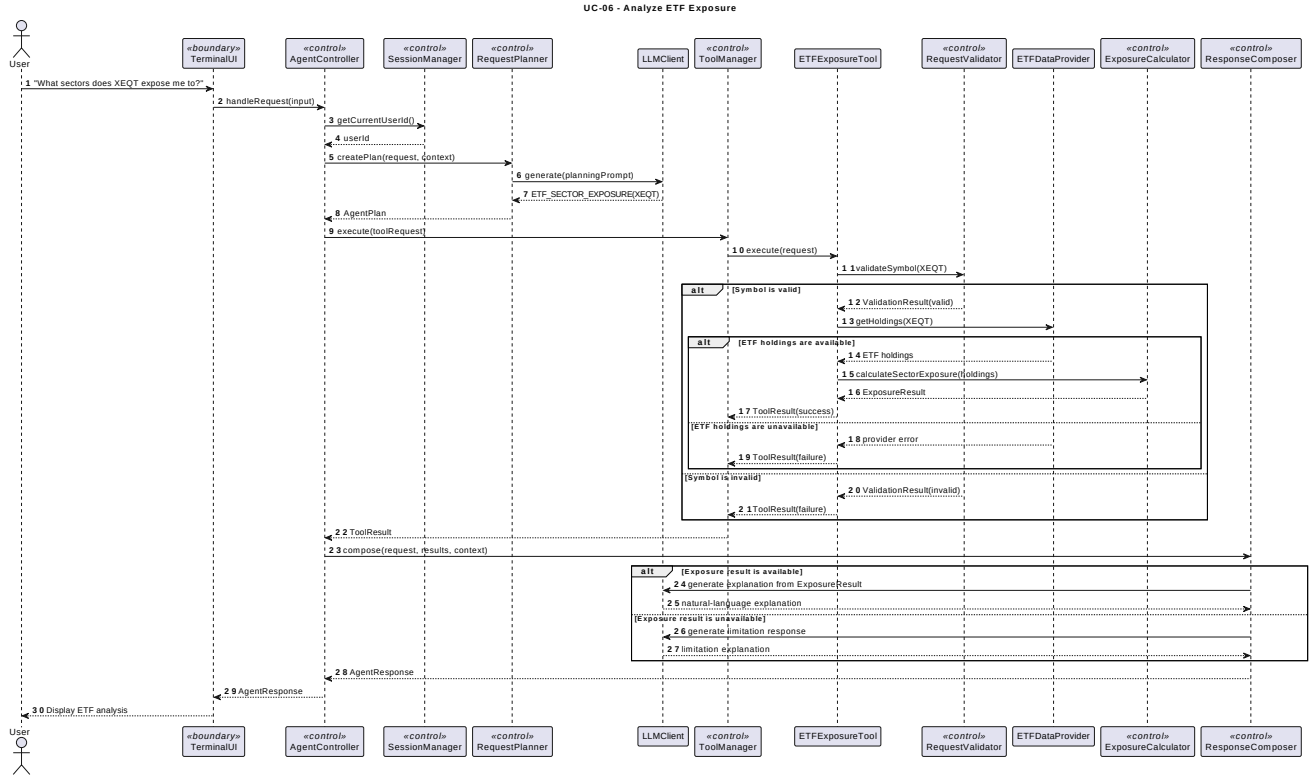


Figure 3: Sequence Diagram for UC-06 – Analyze ETF Exposure

7.3 UC-10 – Explain Recent Market Movement

Figure 4 demonstrates a request that requires multiple independent external tools.

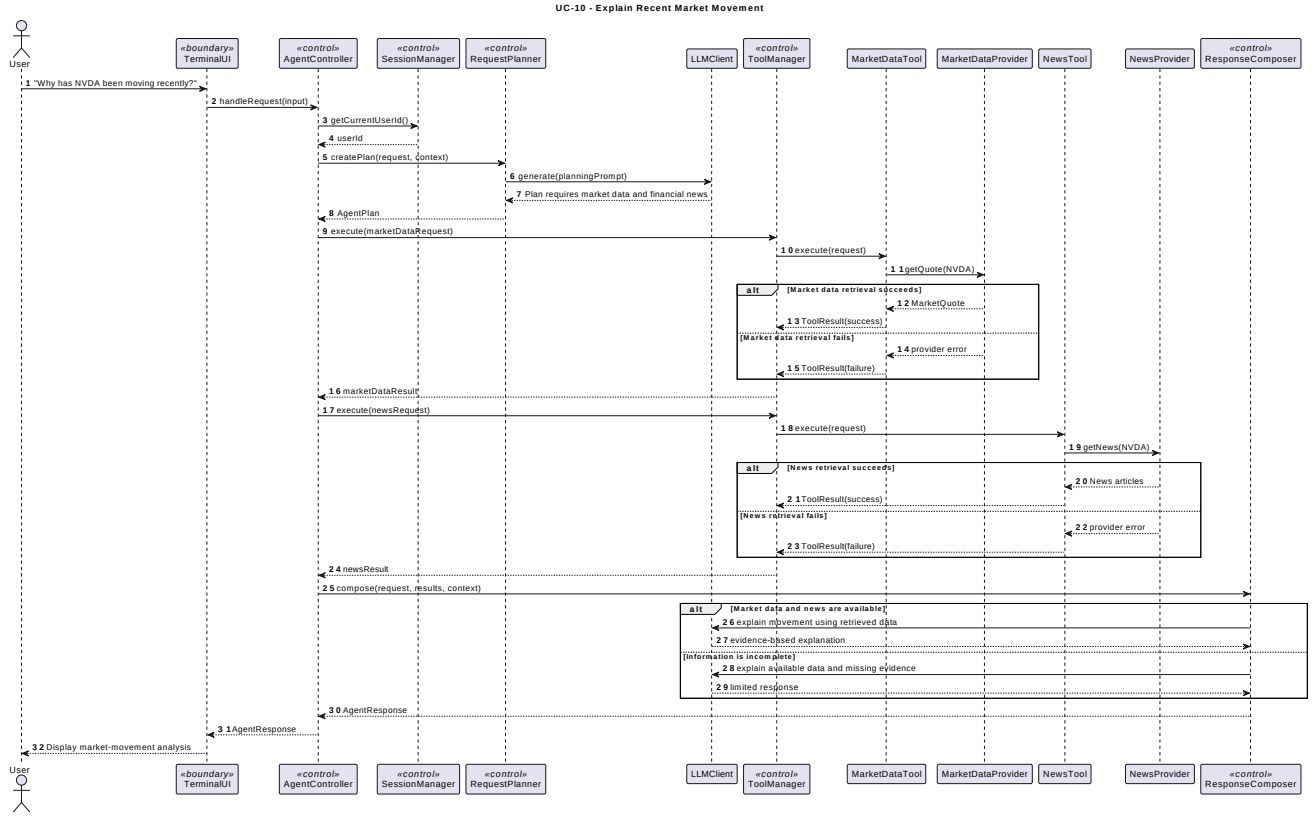


Figure 4: Sequence Diagram for UC-10 – Explain Recent Market Movement

7.4 UC-15 – Analyze Portfolio Exposure

Figure 5 demonstrates a multi-step workflow in which the system loads user-specific portfolio information, retrieves underlying ETF data, performs deterministic calculations, and generates an explanation.

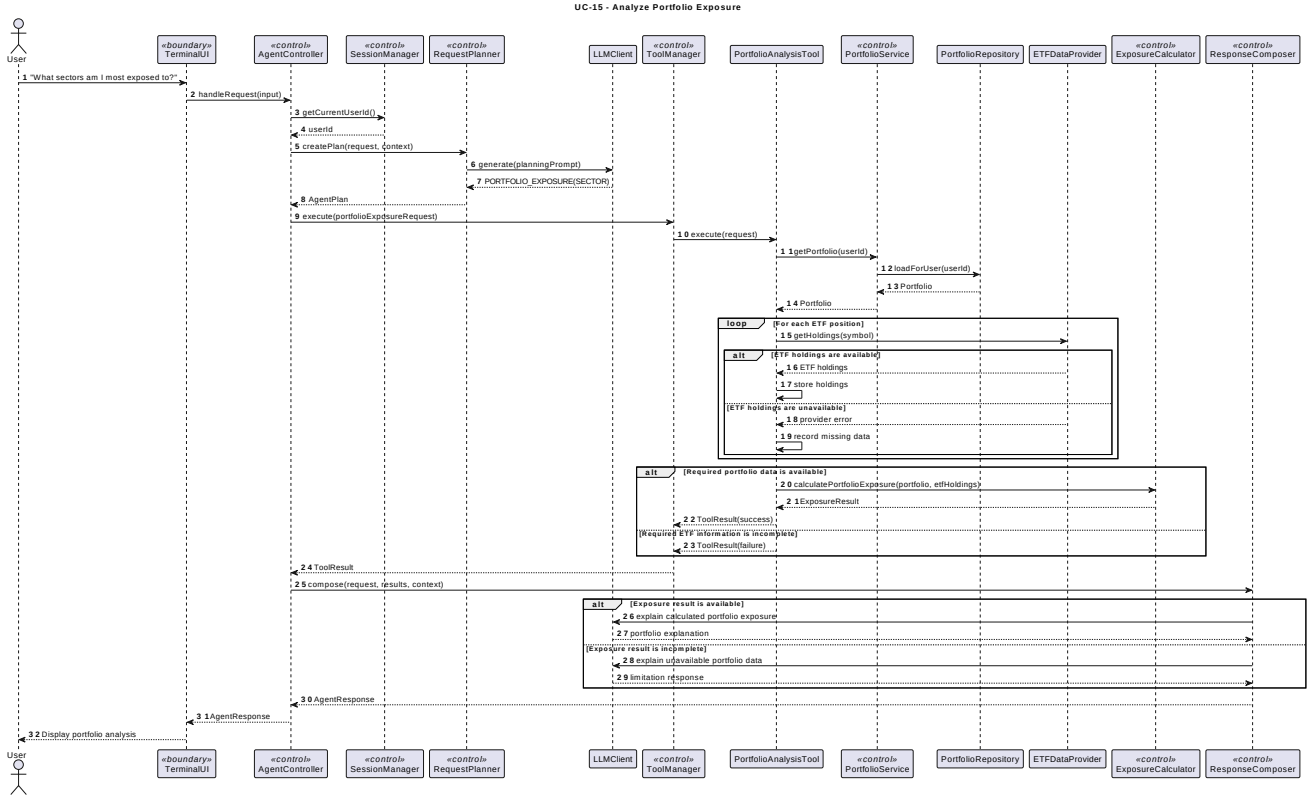
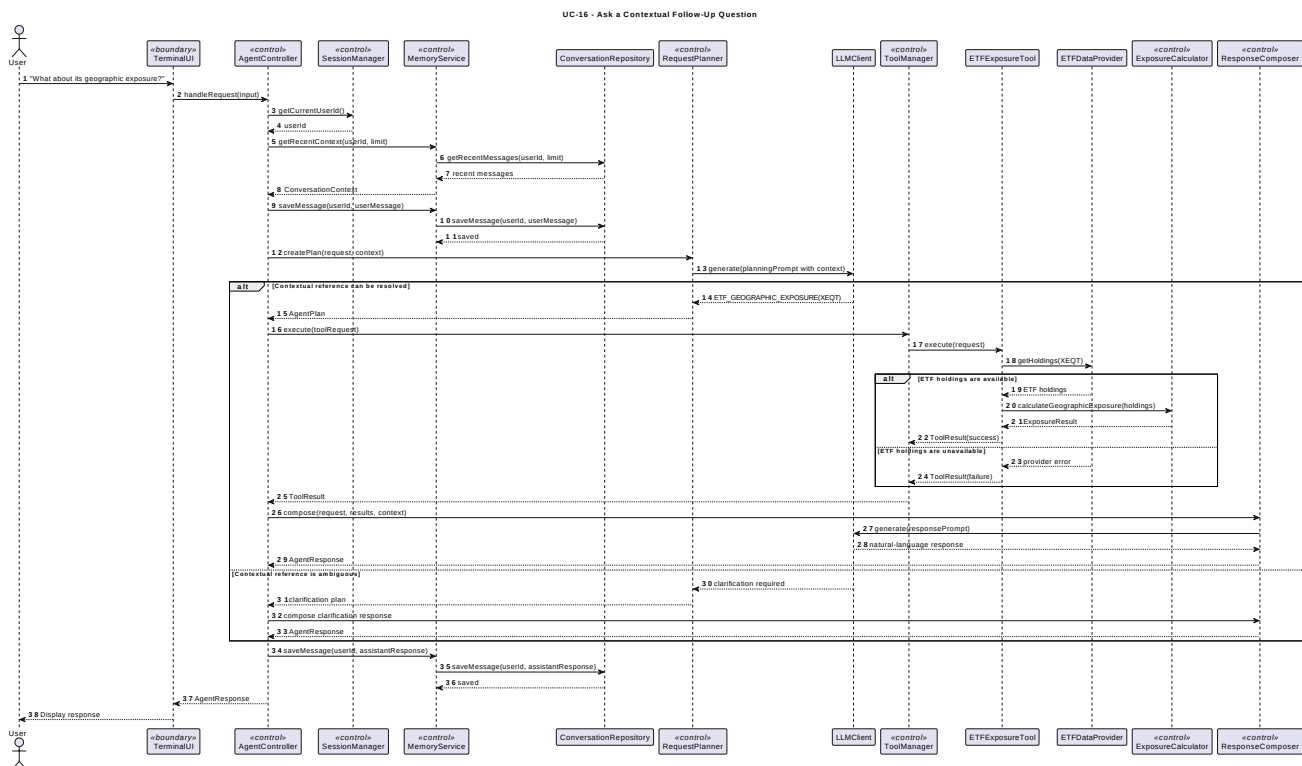


Figure 5: Sequence Diagram for UC-15 – Analyze Portfolio Exposure

7.5 UC-16 – Contextual Follow-Up

Figure 6 demonstrates how user-specific conversation memory participates in resolving a contextual request.



8 Class Design

The architecture is divided conceptually into the following layers:

8.1 User Interface Layer

Principal classes include:

- `UIInterface`;
- `TerminalUI`;
- `CommandParser`; and
- `ParsedCommand`.

8.2 Authentication and Session Layer

Authentication and session management are handled by:

- `AuthService`;
- `PasswordHasher`;
- `BCryptPasswordHasher`;
- `SessionManager`;
- `UserSession`; and
- `UserRepository`.

`AuthService` handles account registration and credential verification.

`PasswordHasher` abstracts password hashing so that authentication logic does not depend directly on one hashing implementation.

`SessionManager` identifies the currently authenticated user and provides the user ID required for user-specific operations.

8.3 Agent Orchestration Layer

The agent layer contains:

- `AgentController`;
- `RequestPlanner`;
- `ToolManager`;
- `ResponseComposer`;
- `PromptBuilder`;
- `LLMClient`; and
- `MemoryService`.

`AgentController` coordinates the complete lifecycle of an agent request.

`RequestPlanner` interprets a request and produces a structured `AgentPlan`.

`ToolManager` selects and executes registered agent tools.

`ResponseComposer` converts structured results into a final response.

8.4 Agent Tool Layer

Agent capabilities implement the common `AgentTool` interface.

Initial tools include:

- `MarketDataTool`;
- `ETFExposureTool`;

- NewsTool;
- PortfolioTool;
- PortfolioAnalysisTool;
- WatchlistTool; and
- ComparisonTool.

This design allows `ToolManager` to depend on the `AgentTool` abstraction rather than on every concrete tool class.

8.5 Domain Service Layer

Deterministic application behavior is implemented by dedicated Java services.

Important classes include:

- ExposureCalculator;
- RequestValidator;
- PortfolioService;
- WatchlistService;
- MemoryService; and
- AuthService.

These components remain separate from LLM reasoning so that their behavior can be verified using conventional automated tests.

8.6 External Provider Layer

External services are accessed through abstractions including:

- MarketDataProvider;
- ETFDataProvider;
- NewsProvider; and
- LLMClient.

Concrete API clients implement these interfaces.

As a result, business logic depends on abstractions instead of directly depending on a specific external vendor.

8.7 Persistence Layer

Persistence uses repository interfaces including:

- UserRepository;
- UserPreferencesRepository;
- PortfolioRepository;
- WatchlistRepository; and
- ConversationRepository.

The planned initial implementations use SQLite.

Each repository operation involving user-owned information accepts or derives a user identifier so that data remains isolated between accounts.

9 UML Class Diagrams

Because the system contains multiple architectural layers, the class model is presented using three complementary diagrams. Together, these diagrams represent the complete proposed software architecture while remaining readable at report scale.

9.1 Core Agent Architecture

Figure 7 shows the user interface, agent orchestration, tool abstraction, deterministic services, and external provider architecture.

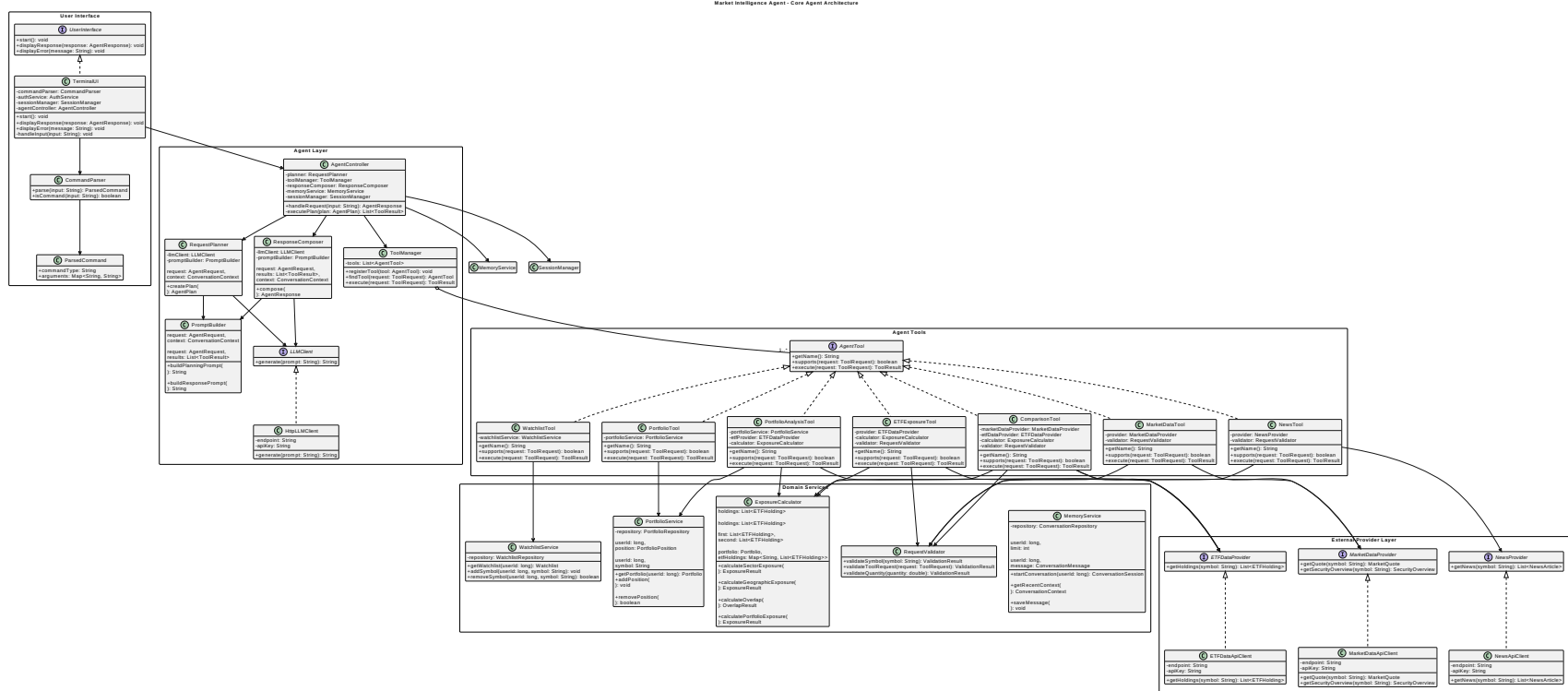
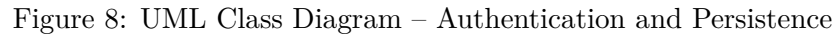


Figure 7: UML Class Diagram – Core Agent Architecture

9.2 Authentication and Persistence

Figure 8 shows authentication, session management, user-specific state, repository abstractions, and SQLite persistence.



9.3 Domain Model

Figure 9 shows the principal agent, market, portfolio, and conversation-domain classes.

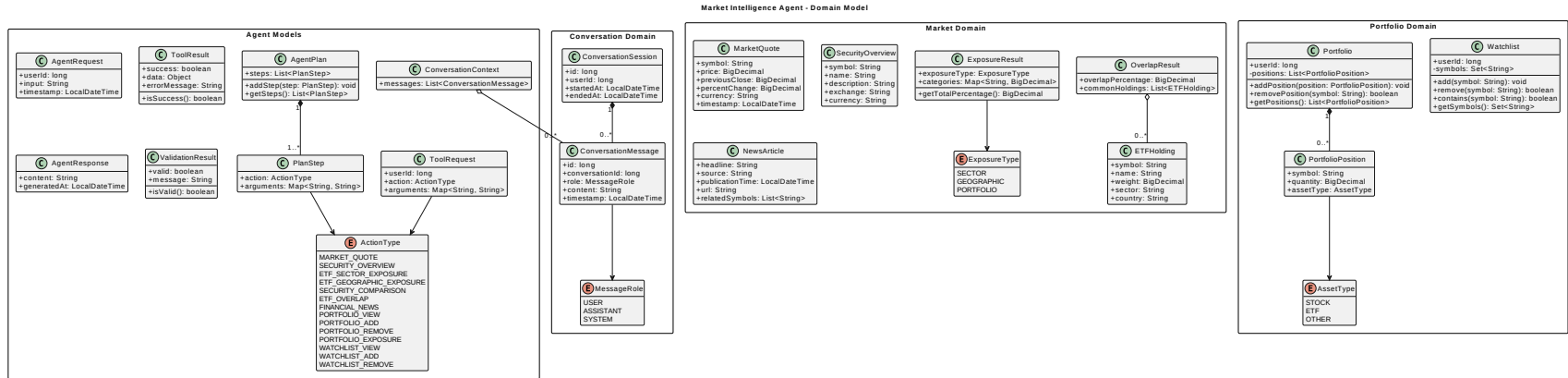


Figure 9: UML Class Diagram – Domain Model

10 Feature-to-Design Mapping

Feature	Main Classes	Responsibilities
User authentication	AuthService, PasswordHasher, UserRepository, SessionManager	Register users, securely verify credentials, and create authenticated user sessions.
Terminal interaction	UserInterface, TerminalUI, CommandParser	Receive natural-language requests and deterministic commands and display results.
Natural-language requests	AgentController, AgentRequest	Receive and coordinate agent requests.
Request planning	RequestPlanner, LLMClient, PromptBuilder, AgentPlan	Interpret requests and produce structured execution plans.
Tool execution	ToolManager, AgentTool, ToolRequest, ToolResult	Locate, execute, and collect results from appropriate tools.
Market-data retrieval	MarketDataTool, MarketDataProvider, MarketQuote	Validate symbols and retrieve market information.
Security overview	MarketDataTool, MarketDataProvider, SecurityOverview	Retrieve descriptive information about supported securities.
ETF analysis	ETFExposureTool, ETFDataProvider, ExposureCalculator	Retrieve ETF holdings and calculate deterministic exposure.
Security comparison	ComparisonTool, MarketDataProvider, ETFDataProvider	Retrieve and compare information for multiple securities.
ETF overlap	ComparisonTool, ExposureCalculator, OverlapResult	Identify common ETF holdings and calculate overlap.
Financial-news retrieval	NewsTool, NewsProvider, NewsArticle	Retrieve structured recent financial-news information.
Market-movement explanation	RequestPlanner, MarketDataTool, NewsTool, ResponseComposer	Combine independently retrieved market and news information into an LLM-generated explanation.
Portfolio management	PortfolioService, PortfolioRepository, PortfolioTool, Portfolio	Store, retrieve, add, and remove portfolio positions for individual users.
Watchlist management	WatchlistService, WatchlistRepository, WatchlistTool, Watchlist	Store and modify user-specific watchlists.
Portfolio exposure	PortfolioAnalysisTool, PortfolioService, ETFDataProvider, ExposureCalculator	Combine direct positions and ETF holdings to calculate portfolio-level exposure.

Feature	Main Classes	Responsibilities
Conversation context	MemoryService, ConversationRepository, ConversationContext	Retrieve recent user-specific conversation information.
Persistent conversation memory	MemoryService, ConversationSession, ConversationMessage, ConversationRepository	Store conversation sessions and messages for each user.
LLM communication	LLMClient, HttpLLMClient, PromptBuilder	Construct and execute requests to the configured LLM.
Response generation	ResponseComposer, LLMClient, AgentResponse	Convert structured tool results into user-facing natural-language explanations.
Input validation	RequestValidator, ValidationResult	Validate symbols, quantities, and structured tool requests before execution.
Persistent storage	DatabaseManager, repository interfaces, SQLite repository implementations	Persist and retrieve user-specific application information.
Failure handling	ToolManager, ToolResult, tool implementations	Represent and propagate failures without substituting fabricated results.

11 Important Design Decisions

11.1 Terminal-Based User Interface

The initial application will use a terminal-based interface rather than a graphical user interface.

This decision keeps the primary implementation effort focused on software architecture, agent behavior, financial calculations, API integration, authentication, persistence, and testing.

The terminal is intended to resemble a professional financial terminal rather than a minimal command prompt.

The `UserInterface` abstraction prevents the core application from depending directly on `TerminalUI`. A graphical interface could therefore be introduced later as another implementation of the same abstraction.

11.2 Separation of Deterministic and LLM Behavior

The LLM will be used where natural-language interpretation or generation is valuable.

Examples include:

- interpreting user intent;
- generating execution plans;
- resolving conversational references;
- summarizing retrieved information; and
- producing natural-language explanations.

The LLM will not perform deterministic business calculations.

Operations such as:

- authentication;
- input validation;
- portfolio state changes;
- ETF exposure calculations;
- overlap calculations;
- persistence;
- API-response mapping; and
- user-data isolation

will be implemented by deterministic Java components.

This provides a clear boundary between conventional application behavior and non-deterministic agent behavior.

11.3 Controlled Agent Execution

The LLM will not directly execute arbitrary Java methods.

Instead, `RequestPlanner` will convert the user's request into a structured `AgentPlan`.

Each `PlanStep` contains an allowed action and structured arguments.

`ToolManager` will map these actions to registered `AgentTool` implementations.

This provides a controlled boundary between LLM-generated planning and application execution.

11.4 Agent Tool Abstraction

All agent capabilities implement the `AgentTool` interface.

The `ToolManager` therefore operates on the abstraction rather than depending directly on every concrete tool.

This design provides:

- polymorphism;
- low coupling;
- extensibility; and
- improved testability.

A new tool can be introduced without requiring the agent controller to be rewritten around the new implementation.

11.5 Provider Abstraction and Dependency Inversion

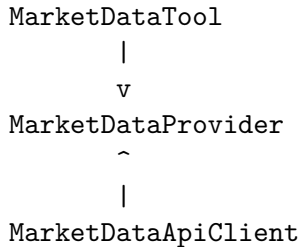
External APIs are represented by Java interfaces such as:

- `MarketDataProvider`;
- `ETFDataProvider`;

- `NewsProvider`; and
- `LLMClient`.

Concrete external clients implement these interfaces.

For example:



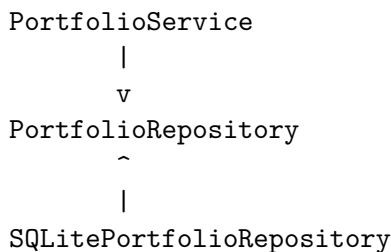
The domain logic therefore depends on an abstraction rather than a particular API vendor.

This supports dependency inversion and allows mock implementations to be used during testing.

11.6 Repository Abstraction

Persistent data is accessed through repository interfaces.

For example:



The domain service is therefore independent of SQLite-specific persistence logic.

This also allows repository implementations to be replaced with test doubles during automated testing.

11.7 Multi-User Data Isolation

Every persistent user-owned resource is associated with a user identifier.

Repository operations therefore use user-specific access patterns such as:

```
Portfolio loadForUser(long userId);
```

```
void saveForUser(
    long userId,
    Portfolio portfolio
);
```

This prevents portfolio, watchlist, preference, and conversation information from becoming shared global state.

11.8 SQLite Persistence

SQLite is planned for the initial persistence implementation.

SQLite provides relational persistence without requiring a separately managed database server, making it appropriate for the scope of the application.

The planned persistent information includes:

- users;
- user preferences;
- portfolios;
- portfolio positions;
- watchlists;
- conversation sessions; and
- conversation messages.

11.9 Password Hashing Abstraction

Password hashing is represented through the `PasswordHasher` interface.

The initial design uses `BCryptPasswordHasher` as its concrete implementation.

Authentication code therefore depends on the hashing abstraction rather than containing hashing logic directly.

11.10 Financial Values and `BigDecimal`

Financial quantities, prices, percentages, ETF weights, and exposure values will use Java's `BigDecimal` rather than binary floating-point types where precision is important.

This allows explicit control over decimal precision and rounding during deterministic financial calculations.

11.11 Constructor-Based Dependency Injection

Major software components will receive dependencies through their constructors.

For example:

```
public MarketDataTool(
    MarketDataProvider provider,
    RequestValidator validator) {

    this.provider = provider;
    this.validator = validator;
}
```

This reduces direct coupling between concrete classes and allows production dependencies to be replaced with mocks or test implementations.

11.12 Structured Failure Results

External-tool failures should not normally be represented only through unhandled exceptions.

Agent tools will return structured `ToolResult` objects containing:

- whether execution succeeded;

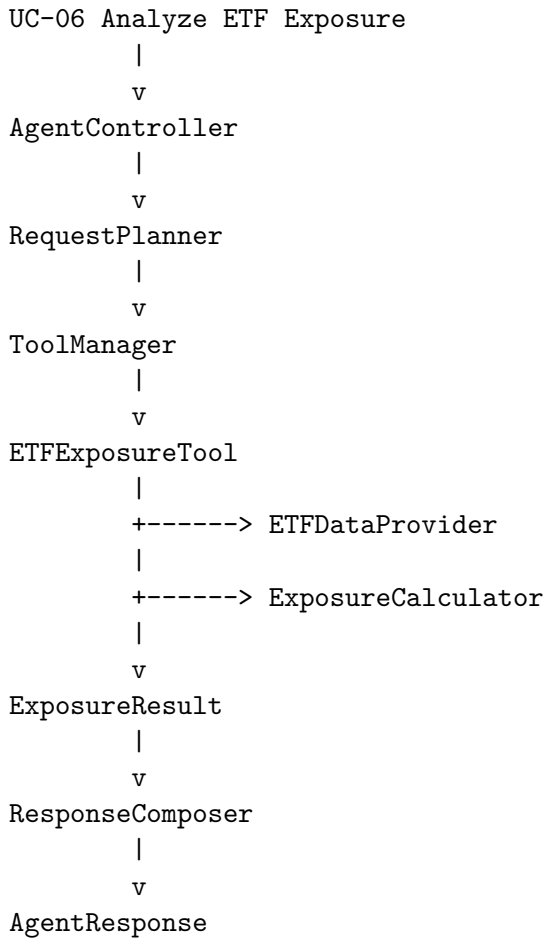
- successfully retrieved data when available; and
- an error message when execution failed.

This allows the agent to explicitly recognize incomplete execution and generate an appropriate response.

12 Design Traceability

The architecture is designed so that major functionality can be traced from the user interaction to the classes responsible for its implementation.

For example, ETF sector analysis follows the conceptual chain:



Similarly, portfolio exposure analysis extends this workflow by retrieving the authenticated user's portfolio before resolving underlying ETF holdings.

This structure will later allow Stage 2 implementation files and Stage 3 tests to be mapped back to the corresponding Stage 1 features and use cases.

13 Stage 1 Summary

The proposed Market Intelligence Agent combines deterministic Java software with an LLM-based planning and response layer.

The application provides sixteen major user-facing interactions including authentication, market-data

retrieval, ETF analysis, financial-news retrieval, portfolio management, watchlist management, portfolio exposure analysis, and contextual conversation.

The system uses a layered architecture separating:

- user interaction;
- authentication and session management;
- agent orchestration;
- tool execution;
- deterministic domain logic;
- external provider access;
- persistent storage; and
- domain models.

Interfaces are used for user interfaces, agent tools, password hashing, external providers, LLM communication, and persistence repositories.

This architecture supports abstraction, encapsulation, separation of concerns, high cohesion, low coupling, dependency inversion, and polymorphism.

The LLM is intentionally prevented from becoming the source of deterministic financial calculations or persistent business state. Java components perform validation, data retrieval, financial calculations, authentication, and persistence, while the LLM is primarily responsible for interpreting natural-language requests, assisting with planning, and explaining structured results.

The resulting design provides a concrete baseline for Stage 2 implementation and enables deterministic software behavior and non-deterministic agent behavior to be tested separately during Stage 3.